Python 21 Core & Extended Modules Reference

Complete executable code reference for all 21 predefined modules and built-in signatures.

Category 1: System & Mathematical Utilities

1. os — Operating System Interface

```
import os

cwd = os.getcwd()                           # Returns current working directory
os.makedirs("data/logs", exist_ok=True)     # Recursively creates directory paths
entries = os.listdir(".")                   # Returns list of entry names in path
full_path = os.path.join(cwd, "data/logs") # Joins path components with separators
exists = os.path.exists(full_path)          # Checks if path refers to existing file/path
env_var = os.environ.get("PATH")            # Accesses system environment variables
os.remove("data/logs/file.txt")             # Deletes specified file
os.rmdir("data/logs")                       # Removes empty directory
```

2. sys — System-Specific Parameters & Functions

```
import sys

cli_args = sys.argv                         # Command line arguments list
module_paths = sys.path                     # Search paths for imported modules
obj_size = sys.getsizeof(["a", "b"])        # Size of an object in bytes
sys.stdout.write("Standard output stream\n")# Writes to standard output buffer
sys.exit(0)                                 # Exits script with optional exit code
```

3. math — Mathematical Functions

```
import math

val_sqrt = math.sqrt(64)                    # Square root calculation
val_pow = math.pow(2, 5)                    # Computes x raised to power y
ceil_val = math.ceil(4.1)                   # Rounds up to nearest integer
floor_val = math.floor(4.9)                 # Rounds down to nearest integer
sin_val = math.sin(math.pi / 2)             # Trigonometric sine (in radians)
cos_val = math.cos(0)                       # Trigonometric cosine
tan_val = math.tan(math.pi / 4)             # Trigonometric tangent
fact = math.factorial(6)                   # Returns n factorial
gcd_val = math.gcd(48, 18)                  # Greatest common divisor
```

4. datetime — Basic Date and Time Types

```
from datetime import datetime

now = datetime.now()                        # Current local date and time
formatted = now.strftime("%Y-%m-%d %H:%M") # Formats datetime object to string
```

Category 2: Time, Calendar & Automation Tools

5. time — Time Access & Conversions

```
import time

ts = time.time()                            # Floating point epoch timestamp in seconds
t1 = time.perf_counter()                    # High-resolution performance clock counter
time.sleep(0.1)                             # Suspends execution for given duration
t2 = time.perf_counter()
struct_time = time.localtime(ts)            # Converts epoch seconds to local struct_time
str_time = time.asctime(struct_time)        # Converts struct_time into readable string
```

6. calendar — General Calendar Utilities

```
import calendar

cal_str = calendar.month(2026, 11)         # Formatted multi-line monthly calendar
is_leap = calendar.isleap(2028)            # Returns True if year is a leap year
w_day = calendar.weekday(2026, 11, 4)     # Weekday index (0=Monday) for date
start_day, total_days = calendar.monthrange(2026, 11) # Starting weekday and total days
```

7. pywhatkit — Automation & Web Action Trigger

```
import pywhatkit as pwk

pwk.sendwhatmsg("+1234567890", "Hello", 15, 30) # Schedules WhatsApp message at hr, min
pwk.sendwhatmsg_instantly("+1234567890", "Fast") # Instantaneous WhatsApp dispatch
pwk.playonyt("Python Programming")            # Searches YouTube & plays top video
pwk.search("Python documentation")            # Executes instant Google web query
pwk.text_to_handwriting("Sample Text", "out.png") # Converts raw text into handwritten image
```

8. webbrowser — Web Browser Controller

```
import webbrowser

webbrowser.open("https://python.org")       # Displays URL using default browser
webbrowser.open_new("https://python.org")   # Opens URL in a new browser window
webbrowser.open_new_tab("https://python.org") # Opens URL in a new page tab
browser_obj = webbrowser.get("firefox")     # Gets controller for specific browser
```

9. winsound — Sound-playing Interface for Windows

```
import winsound

winsound.Beep(2500, 1000)                 # PC speaker beep at frequency (Hz) & time (ms)
```

Category 3: Graphics, Audio, Notifications & Databases

10. PIL (Pillow) — Python Imaging Library

```
from PIL import Image, ImageFilter

img = Image.open("sample.jpg")               # Opens and identifies image file
new_img = Image.new("RGB", (200, 200), "blue") # Creates new image container
resized = img.resize((100, 100))            # Resizes image dimensions
rotated = img.rotate(45)                    # Rotates image by degree angle
filtered = img.filter(ImageFilter.BLUR)    # Applies filter (BLUR, CONTOUR, DETAIL)
img.save("output.png", "PNG")               # Saves graphic output under file path
```

11. pyttsx3 — Offline Text-To-Speech Synthesis

```
import pyttsx3

engine = pyttsx3.init()                     # Initializes text-to-speech engine
engine.setProperty("rate", 150)             # Adjusts rate speed property
volume = engine.getProperty("volume")      # Retrieves current volume configuration
engine.say("Processing complete")          # Queues audio speech command utterance
engine.runAndWait()                         # Processes queued commands synchronously
```

12. plyer — Platform-Independent Notifications

```
from plyer import notification

notification.notify(
    title="Task Alert",
    message="Execution finished successfully.",
    app_icon="icon.ico",
    timeout=10
) # Triggers desktop pop-up alert box
```

13. qrcode — QR Code Generator

```python
import qrcode

quick_qr = qrcode.make("https://example.com") # Quick generator returning PIL image
qr = qrcode.QRCode(version=1, box_size=10, border=4) # Customizable matrix generator
qr.add_data("Payload Data")                  # Attaches string payload data
qr.make(fit=True)                            # Compiles matrix grid
img = qr.make_image()                        # Renders generated QR to downloadable image
```

14. mysql.connector — MySQL Database Connectivity

```python
import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="root", password="secret", database="app_db"
) # Establishes connection pipeline to MySQL server
cursor = conn.cursor()                       # Creates cursor object to run queries
cursor.execute("SELECT * FROM users WHERE id = %s", (1,)) # Executes SQL query
results = cursor.fetchall()
conn.close()
```

Category 4: Web Scraping, Frameworks & Numerical Analytics

15. bs4 (BeautifulSoup) — HTML/XML Parsing

```python
from bs4 import BeautifulSoup

soup = BeautifulSoup("<div class='content'><p id='t'>Text</p></div>", "html.parser")
elem = soup.find("p", attrs={"id": "t"})    # Returns first matching HTML element
all_elems = soup.find_all("p")              # Returns list of all matching HTML elements
attr_val = elem.get("id")                   # Extracts tag property value
text_val = elem.text                        # Strips HTML to plain inner text
selected = soup.select("div.content > p")  # Performs CSS selector queries
```

16. Flask — Micro Web Application Framework

```python
from flask import Flask, jsonify, render_template

app = Flask(__name__)                       # Instantiates core web application context

@app.route("/api/data", methods=["GET"])   # Decorator routing URLs to view handlers
def get_data():
    return jsonify({"status": "success"})  # Serializes dict into JSON response

@app.route("/page")
def show_page():
    return render_template("index.html")   # Renders Jinja2 HTML file template

# app.run(host="0.0.0.0", port=5000)      # Launches local WSGI development server
```

17. numpy — Numerical Scientific Computing

```python
import numpy as np

arr = np.array([1, 2, 3, 4])               # Constructs N-dimensional array wrapper
zeros_arr = np.zeros((2, 3))               # Creates array initialized with all zeros
ones_arr = np.ones((3, 3))                 # Creates array initialized with all ones
range_arr = np.arange(0, 10, 2)            # Evenly spaced elements in interval
lin_arr = np.linspace(0, 1, 5)             # Evenly spaced sequence over range
dot_prod = np.dot(arr, arr)                # Computes matrix dot product
reshaped = arr.reshape((2, 2))             # Reshapes array dimensions
mean_val = np.mean(arr)                    # Mathematical arithmetic mean
std_val = np.std(arr)                      # Standard deviation calculation
sum_val = np.sum(arr)                      # Sums all elements in array
```

18. pandas — Data Analysis & DataFrames

```python
import pandas as pd

df = pd.DataFrame({"A": [1, 2], "B": [3, 4]}) # Constructs 2D labeled tabular data structure
df_csv = pd.read_csv("input.csv")           # Imports CSV file into DataFrame
df.to_csv("output.csv", index=False)        # Exports DataFrame to CSV file
head_rows = df.head(5)                      # Returns first n rows of DataFrame
tail_rows = df.tail(5)                      # Returns last n rows of DataFrame
grouped = df.groupby("A").sum()             # Groups rows by key for group-wise aggregation
```

Category 5: Visualization, Vector Graphics & Unit Testing

19. matplotlib — Comprehensive Visualizations

```python
import matplotlib.pyplot as plt

fig, ax = plt.subplots(2, 2)                # Creates figure and grid of subplot axis objects
plt.plot([1, 2], [3, 4], "r--")           # Renders continuous 2D line plot
plt.bar([1, 2], [10, 20])                 # Generates vertical bar chart
plt.barh([1, 2], [10, 20])                # Generates horizontal bar chart
plt.scatter([1, 2], [5, 6])              # Generates bivariate scatter plot
plt.hist([1, 1, 2, 3, 3, 3])              # Generates frequency histogram
plt.pie([40, 60], labels=["A", "B"], autopct="%1.1f%%") # Circular pie plot
plt.imshow([[1, 2], [3, 4]], cmap="viridis") # Displays 2D density heatmap image
plt.savefig("plot.png")                   # Saves figure image file to disk
plt.show()                                # Displays figure in window
```

20. turtle — Educational Vector Graphics

```python
import turtle

t = turtle.Turtle()                       # Instantiates drawing pen cursor object
screen = turtle.Screen()                  # Instantiates canvas window object
t.forward(100)                            # Moves cursor forward by unit distance
t.backward(50)                            # Moves cursor backward by unit distance
t.right(90)                               # Rotates pen heading clockwise by degrees
t.left(45)                                # Rotates pen heading counterclockwise
t.penup()                                 # Lifts pen off canvas (no trace)
t.pendown()                               # Lowers pen to canvas to draw
t.circle(50)                              # Draws circular arc of specified radius
t.pencolor("red")                         # Sets outline stroke color
t.fillcolor("yellow")                     # Sets internal path fill color
```

21. unittest — Automated Testing Framework

```python
import unittest

class TestModuleCapabilities(unittest.TestCase): # Base class for unit assertions
    def test_assertions(self):
        self.assertEqual(10, 10)          # Asserts equality condition
        self.assertNotEqual(10, 20)       # Asserts inequality condition
        self.assertTrue(True)             # Verifies boolean expression is True
        self.assertFalse(False)           # Verifies boolean expression is False
        with self.assertRaises(ZeroDivisionError): # Validates expected exception error
            val = 1 / 0

if __name__ == "__main__":
    unittest.main()                       # Discovers and executes all test cases
```